01. dstack VMM - VM 생성 과정 상세 분석

목차

1. VMM 아키텍처 개요
2. VM 생성 요청 처리
3. Manifest 생성 과정
4. 작업 디렉토리 준비
5. VM 로딩 과정
6. VM 시작 과정
7. CID 풀 관리
8. 상태 관리

---

1. VMM 아키텍처 개요

1.1 핵심 구조체

```
// vmm/src/app.rs
#[derive(Clone)]
pub struct App {
pub config: Arc<Config>,
pub supervisor: SupervisorClient,
```

```
state: Arc<Mutex<AppState>>,
}

pub(crate) struct AppState {
cid_pool: IdPool<u32>, // CID 할당 풀
vms: HashMap<String, VmState>, // VM 상태 관리
}

#[derive(Clone)]
pub struct VmState {
pub(crate) config: Arc<VmConfig>,
state: VmStateMut,
}
```

1.2 VM 설정 구조체

```
// vmm/src/app.rs
#[derive(Deserialize, Serialize, Clone, Builder, Debug)]
pub struct Manifest {
pub id: String, // VM 고유 ID
pub name: String, // 사용자 지정 이름
pub app_id: String, // 애플리케이션 ID
pub vcpu: u32, // 가상 CPU 수
```

```
pub memory: u32, // 메모리 크기 (MB)
pub disk_size: u32, // 디스크 크기 (GB)
pub image: String, // OS 이미지 이름
pub port_map: Vec<PortMapping>, // 포트 매핑 설정
pub created_at_ms: u64, // 생성 시간
pub hugepages: bool, // 대용량 페이지 사용 여부
pub pin_numa: bool, // NUMA 핀 설정
pub gpus: Option<GpuConfig>, // GPU 설정
pub kms_urls: Vec<String>, // KMS 서버 URL 목록
pub gateway_urls: Vec<String>, // Gateway 서버 URL 목록
}
```

---

2. VM 생성 요청 처리

2.1 RPC 엔드포인트

```
// vmm/src/main_service.rs
impl VmmRpc for RpcHandler {
async fn create_vm(self, request: VmConfiguration) -> Result<Id> {
// 1. 이름 유효성 검사
validate_label(&request.name)?;
```

```
// 2. 포트 매핑 검증
let pm_cfg = &self.app.config.cvm.port_mapping;
if !(request.ports.is_empty() || pm_cfg.enabled) {
bail!("Port mapping is disabled");
}
// 3. 포트 매핑 구성
let port_map = request
.ports
.iter()
.map(|p| {
let from = p.host_port.try_into().context("Invalid host port")?;
let to = p.vm_port.try_into().context("Invalid vm port")?;
if !pm_cfg.is_allowed(&p.protocol, from) {
bail!("Port mapping is not allowed for {}:{}", p.protocol, from);
}
let protocol = p.protocol.parse().context("Invalid protocol")?;
let address = if !p.host_address.is_empty() {
p.host_address.parse().context("Invalid host address")?
} else {
pm_cfg.address
};
Ok(PortMapping {
address,
```

```
protocol,
from,
to,
})
})
.collect::<Result<Vec<_>>>()?;


// 4. App ID 결정
let app_id = match &request.app_id {
Some(id) => id.strip_prefix("0x").unwrap_or(id).to_lowercase(),
None => app_id_of(&request.compose_file),
};
// 5. 고유 VM ID 생성
let id = uuid::Uuid::new_v4().to_string();
let now = SystemTime::now()
.duration_since(UNIX_EPOCH)
.unwrap_or_default()
.as_millis() as u64;
// 6. GPU 설정 처리
let gpus = match &request.gpus {
Some(gpus) => self.resolve_gpus(gpus)?,
None => GpuConfig::default(),
};
```

```
// 7. Manifest 생성
let manifest = Manifest::builder()
.id(id.clone())
.name(request.name.clone())
.app_id(app_id.clone())
.image(request.image.clone())
.vcpu(request.vcpu)
.memory(request.memory)
.disk_size(request.disk_size)
.port_map(port_map)
.created_at_ms(now)
.hugepages(request.hugepages)
.pin_numa(request.pin_numa)
.gpus(gpus)
.kms_urls(request.kms_urls.clone())
.gateway_urls(request.gateway_urls.clone())
.build();
// 8. 작업 디렉토리 준비
let vm_work_dir = self.app.work_dir(&id);
vm_work_dir
.put_manifest(&manifest)
.context("Failed to write manifest")?;
let work_dir = self.prepare_work_dir(&id, &request, &app_id)?;
```

```
// 9. 시작 상태 설정
if  let Err(err) = vm_work_dir.set_started(!request.stopped) {
warn!("Failed to set started: {}", err);
}


// 10. VM 로딩 및 시작
let  result = self
.app
.load_vm(&work_dir, &Default::default(), false)
.await
.context("Failed to load VM");
let  result = match  result {
Ok(()) => {
if !request.stopped {
self.app.start_vm(&id).await
} else {
Ok(())
}
}
Err(err) => Err(err),
};
// 11. 실패 시 정리
if  let Err(err) = result {
```

```
if  let Err(err) = fs::remove_dir_all(&work_dir) {
warn!("Failed to remove work dir: {}", err);
}
return Err(err);
}

Ok(Id { id })
}
}
```

2.2 App ID 생성 함수

```
// vmm/src/main_service.rs
fn  app_id_of(compose_file: &str) -> String {
fn  truncate40(s: &str) -> &str {
if  s.len() > 40 {
&s[..40]
} else {
s
}
}
truncate40(&hex_sha256(compose_file)).to_string()
}
```

```rust
fn hex_sha256(data: &str) -> String {
use sha2::Digest;
let mut hasher = sha2::Sha256::new();
hasher.update(data);
hex::encode(hasher.finalize())
}
```

---

3. Manifest 생성 과정

3.1 Builder 패턴 사용

```rust
// vmm/src/app.rs에서 Builder 매크로 사용
#[derive(Deserialize, Serialize, Clone, Builder, Debug)]
pub struct Manifest {
// ... 필드들
}

// 실제 사용 예시 (main_service.rs)
let manifest = Manifest::builder()
.id(id.clone())
.name(request.name.clone())
.app_id(app_id.clone())
```

```
.image(request.image.clone())
.vcpu(request.vcpu)
.memory(request.memory)
.disk_size(request.disk_size)
.port_map(port_map)
.created_at_ms(now)
.hugepages(request.hugepages)
.pin_numa(request.pin_numa)
.gpus(gpus)
.kms_urls(request.kms_urls.clone())
.gateway_urls(request.gateway_urls.clone())
.build();
```

3.2 GPU 설정 처리

```
// vmm/src/main_service.rs
fn resolve_gpus(gpu_cfg: &rpc::GpuConfig) -> Result<GpuConfig> {
match gpu_cfg.attach_mode.as_str() {
"listed" => {
// 지정된 GPU 목록 사용
let gpus = gpu_cfg
.gpus
.iter()
```

```
.map(|g| GpuSpec {
slot: g.slot.clone(),
})
.collect();

Ok(GpuConfig {
attach_mode: AttachMode::Listed,
gpus,
bridges: Vec::new(),
})
}
"all" => {
// 모든 NVIDIA GPU 자동 탐지
let devices = lspci::lspci_filtered(|dev| {
// NVIDIA 벤더 ID (10de) 확인
dev.vendor_id == "10de"
})
.context("Failed to list PCI devices")?;

let mut gpus = Vec::new();
let mut bridges = Vec::new();

for dev in devices {
```

```
if dev.class.contains("3D controller") {
gpus.push(GpuSpec { slot: dev.slot });
} else if dev.class.contains("Bridge") {
bridges.push(GpuSpec { slot: dev.slot });
}
}
Ok(GpuConfig {
attach_mode: AttachMode::All,
gpus,
bridges,
})
}
_ => bail!("Invalid GPU attach mode: {}", gpu_cfg.attach_mode),
}
}
```

---

4. 작업 디렉토리 준비

4.1 디렉토리 구조 생성

```
// vmm/src/app.rs
pub(crate) fn prepare_work_dir(
```

```
&self,
id: &str,
req: &VmConfiguration,
app_id: &str,
) -> Result<VmWorkDir> {
let  work_dir = self.work_dir(id);
let  shared_dir = work_dir.join("shared");
// 1. 공유 디렉토리 생성
fs::create_dir_all(&shared_dir).context("Failed to create shared directory")?;
// 2. docker-compose.yaml 파일 작성
fs::write(shared_dir.join(APP_COMPOSE), &req.compose_file)
.context("Failed to write compose file")?;
// 3. 암호화된 환경 변수 파일 작성 (있는 경우)
if !req.encrypted_env.is_empty() {
fs::write(shared_dir.join(ENCRYPTED_ENV), &req.encrypted_env)
.context("Failed to write encrypted env")?;
}
// 4. 사용자 설정 파일 작성 (있는 경우)
if !req.user_config.is_empty() {
fs::write(shared_dir.join(USER_CONFIG), &req.user_config)
.context("Failed to write user config")?;
}
// 5. 인스턴스 정보 파일 작성
```

```
if !app_id.is_empty() {
let instance_info = json!({
"app_id": app_id,
});
fs::write(
shared_dir.join(INSTANCE_INFO),
serde_json::to_string(&instance_info)?,
)
.context("Failed to write vm config")?;
}
Ok(work_dir)
}
```

4.2 파일 상수 정의

```
// dstack_types/src/shared_filenames.rs에서 정의
pub const APP_COMPOSE: &str = "app-compose.json";
pub const ENCRYPTED_ENV: &str = "encrypted-env";
pub const USER_CONFIG: &str = "user-config.json";
pub const INSTANCE_INFO: &str = "instance-info.json";
pub const SYS_CONFIG: &str = "sys-config.json";
```

---

5. VM 로딩 과정

5.1 load_vm 함수

```
// vmm/src/app.rs
pub  async  fn  load_vm(
&self,
work_dir: impl AsRef<Path>,
cids_assigned: &HashMap<String, u32>,
auto_start: bool,
) -> Result<()> {
let  vm_work_dir = VmWorkDir::new(work_dir.as_ref());
// 1. Manifest 읽기
let  manifest = vm_work_dir.manifest().context("Failed to read manifest")?;
// 2. 이미지 이름 유효성 검사
if  manifest.image.len() > 64
|| manifest.image.contains("..")
|| !manifest
.image
.chars()
.all(|c| c.is_alphanumeric() || c == '_' || c == '-' || c == '.')
{
bail!("Invalid image name");
```

```
}

// 3. 이미지 로딩

let image_path = self.config.image_path.join(&manifest.image);

let image = Image::load(&image_path).context("Failed to load image")?;

let vm_id = manifest.id.clone();

// 4. App Compose 파일 읽기

let app_compose = vm_work_dir

.app_compose()

.context("Failed to read compose file")?;

// 5. CID 할당 및 VM 설정 생성

{

let mut states = self.lock();

let cid = states

.get(&vm_id)

.map(|vm| vm.config.cid)

.or_else(|| cids_assigned.get(&vm_id).cloned())

.or_else(|| states.cid_pool.allocate())

.context("CID pool exhausted")?;

let vm_config = VmConfig {

manifest,

image,

cid,

networking: self.config.networking.clone(),
```

```
workdir: vm_work_dir.path().to_path_buf(),

gateway_enabled: app_compose.gateway_enabled(),

};

// 6. VM 상태 업데이트 또는 생성

match  states.get_mut(&vm_id) {

Some(vm) => {

vm.config = vm_config.into();

}

None => {

states.add(VmState::new(vm_config));

}

}

};

// 7. 자동 시작 (설정된 경우)

if  auto_start && vm_work_dir.started().unwrap_or_default() {

self.start_vm(&vm_id).await?;

}

Ok(())

}
```

5.2 이미지 로딩

```
// vmm/src/image.rs (추정)
```

```rust
impl Image {
pub fn load(path: &Path) -> Result<Self> {
// 이미지 메타데이터 로딩
// 이미지 무결성 검증
// 이미지 정보 파싱
}
}
```

---

6. VM 시작 과정

6.1 start_vm 함수

```rust
// vmm/src/app.rs
pub async fn start_vm(&self, id: &str) -> Result<()> {
// 1. 동적 설정 동기화
self.sync_dynamic_config(id)?;
// 2. 실행 상태 확인
let is_running = self
.supervisor
.info(id)
.await?
.is_some_and(|info| info.state.status.is_running());
// 3. 시작 상태 설정
```

```
self.set_started(id, true)?;

// 4. VM 설정 가져오기 및 상태 업데이트

let vm_config = {

let mut state = self.lock();

let vm_state = state.get_mut(id).context("VM not found")?;

// 구버전 이미지 호환성 처리

if vm_state.config.image.info.shared_ro {

vm_state.state.start(is_running);

} else {

vm_state.state.reset_na();

}

vm_state.config.clone()

};

// 5. 실제 VM 시작 (아직 실행 중이 아닌 경우)

if !is_running {

let work_dir = self.work_dir(id);

// 기존 소켓 파일 정리

for path in [work_dir.serial_pty(), work_dir.qmp_socket()] {

if path.symlink_metadata().is_ok() {

fs::remove_file(path)?;

}

}
```

```
// GPU 할당
let devices = self.try_allocate_gpus(&vm_config.manifest)?;
// QEMU 프로세스 설정 생성
let process_config = vm_config.config_qemu(&work_dir, &self.config.cvm, &device
// Supervisor를 통한 프로세스 배포
self.supervisor
.deploy(process_config)
.await
.with_context(|| format!("Failed to start VM {id}"))?;


// GPU 디바이스 정보 저장
let mut state = self.lock();
let vm_state = state.get_mut(id).context("VM not found")?;
vm_state.state.devices = devices;
}
Ok(())
}
```

6.2 동적 설정 동기화

```
// vmm/src/app.rs
pub(crate) fn sync_dynamic_config(&self, id: &str) -> Result<()> {
let work_dir = self.work_dir(id);
```

```
let shared_dir = self.shared_dir(id);
let manifest = work_dir.manifest().context("Failed to read manifest")?;
let cfg = &self.config;
// 1. 이미지 정보 로딩
let image_path = cfg.image_path.join(&manifest.image);
let image = Image::load(image_path).context("Failed to load image info")?;
let img_ver = image.info.version_tuple().unwrap_or((0, 0, 0));
// 2. KMS/Gateway URL 결정
let kms_urls = if manifest.kms_urls.is_empty() {
cfg.cvm.kms_urls.clone()
} else {
manifest.kms_urls.clone()
};
let gateway_urls = if manifest.gateway_urls.is_empty() {
cfg.cvm.gateway_urls.clone()
} else {
manifest.gateway_urls.clone()
};
// 3. 버전별 시스템 설정 생성
let sys_config = if img_ver >= (0, 5, 0) {
// 최신 버전 설정
let os_image_hash = hex::decode(image.digest.unwrap_or_default())
.context("Failed to decode image digest")?;
```

```
let  gpus = manifest.gpus.unwrap_or_default();

let  vm_config = serde_json::to_string(&dstack_types::VmConfig {

spec_version: 1,

os_image_hash,

cpu_count: manifest.vcpu,

memory_size: manifest.memory as u64 * 1024 * 1024,

qemu_single_pass_add_pages: cfg.cvm.qemu_single_pass_add_pages,

pic: cfg.cvm.qemu_pic,

pci_hole64_size: cfg.cvm.qemu_pci_hole64_size,

hugepages: manifest.hugepages,

num_gpus: gpus.gpus.len() as u32,

num_nvswitches: gpus.bridges.len() as u32,

hotplug_off: cfg.cvm.qemu_hotplug_off,

})?;

json!({

"kms_urls": kms_urls,

"gateway_urls": gateway_urls,

"pccs_url": cfg.cvm.pccs_url,

"docker_registry": cfg.cvm.docker_registry,

"host_api_url": format!("vsock://2:{}/api", cfg.host_api.port),

"vm_config": vm_config,

})

} else  if  img_ver >= (0, 4, 2) {
```

```
// 중간 버전 설정
json!({
"kms_urls": kms_urls,
"gateway_urls": gateway_urls,
"pccs_url": cfg.cvm.pccs_url,
"docker_registry": cfg.cvm.docker_registry,
"host_api_url": format!("vsock://2:{}/api", cfg.host_api.port),
})
} else {
// 구버전 호환성 설정
// ... 구버전 설정 로직
};
// 4. 설정 파일 작성
let  sys_config_str = serde_json::to_string(&sys_config)
.context("Failed to serialize vm config")?;
let  config_file = if  img_ver >= (0, 4, 0) {
SYS_CONFIG
} else {
compat_v3::SYS_CONFIG
};
fs::write(shared_dir.join(config_file), sys_config_str)
.context("Failed to write vm config")?;
// 5. 구버전 호환성 처리
```

```
if img_ver < (0, 4, 0) {
// 암호화된 환경 변수 파일 동기화
// 인증서 파일 복사
// ... 구버전 호환성 로직
}
Ok(())
}
```

---

7. CID 풀 관리

7.1 CID 풀 초기화

```
// vmm/src/app.rs
impl App {
pub fn new(config: Config, supervisor: SupervisorClient) -> Self {
let cid_start = config.cvm.cid_start;
let cid_end = cid_start.saturating_add(config.cvm.cid_pool_size);
let cid_pool = IdPool::new(cid_start, cid_end);
Self {
supervisor: supervisor.clone(),
state: Arc::new(Mutex::new(AppState {
cid_pool,
```

```
vms: HashMap::new(),
})),
config: Arc::new(config),
}
}
}
```

7.2 CID 할당 과정

```
// vmm/src/app.rs의 load_vm 함수 내부
let cid = states
.get(&vm_id) // 1. 기존 VM의 CID 확인
.map(|vm| vm.config.cid)
.or_else(|| cids_assigned.get(&vm_id).cloned()) // 2. 할당된 CID 확인
.or_else(|| states.cid_pool.allocate()) // 3. 새 CID 할당
.context("CID pool exhausted")?;
```

7.3 CID 해제

```
// vmm/src/app.rs
pub async fn remove_vm(&self, id: &str) -> Result<()> {
// ... VM 중지 로직 ...
{
```

```
let mut state = self.lock();
if let Some(vm_state) = state.remove(id) {
// CID 풀에 반환
state.cid_pool.free(vm_state.config.cid);
}
}
// VM 디렉토리 삭제
let vm_path = self.work_dir(id);
fs::remove_dir_all(&vm_path).context("Failed to remove VM directory")?;
Ok(())
}
```

---

8. 상태 관리

8.1 VM 상태 구조체

```
// vmm/src/app.rs
#[derive(Debug, Clone, Default)]
struct VmStateMut {
boot_progress: String, // 부팅 진행 상태
boot_error: String, // 부팅 오류 메시지
shutdown_progress: String, // 종료 진행 상태
devices: GpuConfig, // 할당된 GPU 디바이스
```

```
}

impl VmStateMut {
pub fn start(&mut self, already_running: bool) {
self.boot_progress = if already_running {
"running".to_string()
} else {
"booting".to_string()
};
self.boot_error.clear();
self.shutdown_progress.clear();
}

pub fn reset_na(&mut self) {
self.boot_progress = "N/A".to_string();
self.shutdown_progress = "N/A".to_string();
self.boot_error.clear();
}
}
```

8.2 VM 이벤트 처리

```
// vmm/src/app.rs
```

```
pub(crate) fn  vm_event_report(&self, cid: u32, event: &str, body: String) -> Re

info!(cid, event, "VM event");

if  body.len() > 1024 * 4 {

error!("Event body too large, skipping");

return Ok(());

}

let  mut  state = self.lock();

let Some(vm) = state.vms.values_mut().find(|vm| vm.config.cid == cid) else {

bail!("VM not found");

};

match  event {

"boot.progress" => {

vm.state.boot_progress = body;

}

"boot.error" => {

vm.state.boot_error = body;

}

"shutdown.progress" => {

if  body == "powering off" {

self.set_started(&vm.config.manifest.id, false)?;

}

vm.state.shutdown_progress = body;

}
```

```
"instance.info" => {
let  workdir = VmWorkDir::new(vm.config.workdir.clone());
let  instancd_info_path = workdir.instance_info_path();
safe_write::safe_write(&instancd_info_path, &body)?;
}
_ => {
error!("Guest reported unknown event: {event}");
}
}
Ok(())
}
```

8.3 자동 재시작 메커니즘

```
// vmm/src/app.rs
pub(crate) async  fn  try_restart_exited_vms(&self) -> Result<()> {
// 1. 현재 실행 중인 VM 목록 조회
let  running_vms = self
.supervisor
.list()
.await
.context("Failed to list VMs")?
.iter()
```

```
.filter(|v| v.state.status.is_running())
.map(|v| v.config.id.clone())
.collect::<BTreeSet<_>>();
// 2. 시작되어야 하지만 실행되지 않은 VM 찾기
let exited_vms = self
.lock()
.iter_vms()
.filter(|vm| {
let workdir = self.work_dir(&vm.config.manifest.id);
let started = workdir.started().unwrap_or(false);
started && !running_vms.contains(&vm.config.manifest.id)
})
.map(|vm| vm.config.manifest.id.clone())
.collect::<Vec<_>>();
// 3. 종료된 VM들 재시작
for id in exited_vms {
info!("Restarting VM {id}");
self.start_vm(&id).await?;
}
Ok(())
}
```

요약

dstack VMM의 VM 생성 과정은 다음과 같은 단계로 구성됩니다:

1. **요청 검증**: 이름, 포트 매핑, GPU 설정 등 유효성 검사
2. **Manifest 생성**: VM 메타데이터와 설정 정보 구조화
3. **작업 디렉토리 준비**: 필요한 파일들(compose, 환경변수 등) 생성
4. **VM 로딩**: 이미지 로딩, CID 할당, 상태 관리 구조 생성
5. **VM 시작**: 동적 설정 동기화, QEMU 프로세스 시작
6. **상태 관리**: 부팅 진행 상태, 이벤트 처리, 자동 재시작

각 단계는 실제 함수명과 구조체를 사용하여 구현되며, 오류 처리와 리소스 관리가 체계적으로 이루어집니다.